

Particle Model

30th January 2012

Copyright ©2012 Mantis Numerics Limited
1 Oakwood Nook,
Leeds LS8 2JA,
United Kingdom.

1 Introduction

There are many problems in which it is necessary to deal with the interaction of either solid or liquid particles with a fluid flow. If the particles are large enough for their velocity to be significantly different from that of the fluid, then it is most efficient to integrate the equation of motion for each particle. On the other hand, very small particles are likely to move at their terminal velocity, which depends upon the fluid acceleration. In that case the particles are best described by a distribution function $f(m, \mathbf{r}, t)$ such that $f\mathrm{d}m$ is the number density of particles with masses in the range m to $m + \mathrm{d}m$.

These two descriptions are complementary since, if the particles are large, their number density must be small so that a Lagrangian calculation is not too expensive. Furthermore, their inertia is significant, which means that the equation for the particle motion is not stiff. Small particles are likely to be very numerous if there is a significant mass of particles and the particle equation of motion is stiff due to the small particle mass. One could, of course cover both cases with a distribution function that depends upon the particle velocity, but this would lead to a seven dimensional problem for three dimensional unsteady flows and would be prohibitively expensive to compute.

2 Physical Model

The physics used in the model similar to that discussed by Boyson & Swithenbank (1979). Any such model consists of two elements: an equation of motion and equations describing changes in particle properties such as mass and temperature.

2.1 Equation of Motion

The particle equation of motion is

$$\frac{\mathrm{d}\mathbf{v}_p}{\mathrm{d}t} = -\frac{3}{16}\frac{\mu}{\rho_p r_p^2} C_D(\mathbf{v}_p - \mathbf{v}_f) + \left(1 - \frac{\rho_f}{\rho_p}\right)\mathbf{g} + \frac{3}{r_p}(\mathbf{v}_f - \mathbf{v}_p)\frac{\mathrm{d}r_p}{\mathrm{d}t}. \tag{1}$$

Here and throughout, suffices p and f refer to the particles and fluid respectively. r_p and $\mathbf{v}_p$ are the particle radius and velocity, μ is the fluid dynamical viscosity, $\mathbf{g}$ is the acceleration due to gravity and

$$C_D = 24\left(1 + \frac{R_e^{2/3}}{6}\right), \tag{2}$$

is the drag coefficient. Here

$$R_e = 2\frac{r_p \rho_f}{\mu}|\mathbf{v}_p - \mathbf{v}_f|, \tag{3}$$

is the Reynolds number based on the particle diameter.

Note that we have assumed that the particles have a uniform density i.e. the particle mass is given by

$$m = \frac{4\pi}{3}\rho_p r_p^3, \tag{4}$$

Boyson & Swithenbank (1979) describe more complex equations that include inertial drag and the Bassett force, but these do not seem to have been used in the calculations described in Dodge & Schwalb (1989) Note that Boyson and Swithenbank (1979) neglect the last term in equation (1), but it is required to ensure conservation of total momentum.

2.2 Particle Evolution

In order to compute the thermal evolution of the particles is convenient to use the thermal energy of a particle

$$e_p = mC_pT_p \tag{5}$$

where C_p is the specific heat of the particle and T_p is its temperature.

The interaction between the particles and fluid is complicated, but the thermal energy of a particle can be assumed to be governed by an equation of the form

$$\frac{\mathrm{d}e_p}{\mathrm{d}t} = s_e, \tag{6}$$

where $s_e = s(T_p, m, \rho_f, T_f, p_v)$ describes the transfer of thermal energy between the particle and the fluid. Here T_f, p_v are the temperature of the fluid and the vapour pressure in the fluid.

Similarly, the particle mass is governed by

$$\frac{\mathrm{d}m}{\mathrm{d}t} = s_m \tag{7}$$

where $s_T = s(T_p, m, \rho_f, T_f, p_v)$ accounts for evaporation or condensation. Note that these equations simply account for changes in the particles as they move. Particle splitting or coagulation introduce additional terms that will be discussed later.

3 Numerical Scheme

The behaviour of the particles is described by the ordinary differential equations (1), (6) and (7), with the possible inclusion of a model for particle splitting or coagulation. The way in which these are incorporated into the code depends upon whether individual particles are being tracked, or being described by a distribution function.

3.1 Individual Particles

In this case we just need to integrate equations (1), (6) and (7) for each particle using interpolated values of the fluid variables at the particle position. This is done using a second order Runge-Kutta algorithm. Particle splitting or coagulation is modelled simply by splitting or merging individual particles.

Unless the particles are very large, we do not expect them to move significantly faster than the fastest fluid, which means that is impossible for them to move through more than one cell in a time step. However, for very small particles, it is possible for the acceleration timescale to become very short compared with the fluid timestep, which means that the particles move at their terminal velocity. An explicit integration of (1) would then require a very small timestep, which is clearly inefficient. In such cases it is better to assume that the particles move at their terminal velocity i.e. to neglect their inertia. One can then obtain an equation that relates the particle velocity to the fluid acceleration.

If we define a relaxation time by

$$\tau_d = \frac{16}{3}\frac{\rho_p r_p^2 C_D}{\mu}, \tag{8}$$

then we can write the equation of motion (1) in the form

$$\frac{\mathrm{d}\mathbf{v}_p}{\mathrm{d}t} = -\frac{1}{\tau_d}(\mathbf{v}_p - \mathbf{v}_f) + \left(1 - \frac{\rho_f}{\rho_p}\right)\mathbf{g} + \frac{\dot{m}}{m}(\mathbf{v}_f - \mathbf{v}_p),$$

It is convenient to rewrite this in the form

$$\frac{\mathrm{d}}{\mathrm{d}t}(\mathbf{v}_p - \mathbf{v}_f) = -\frac{1}{\tau_d}(\mathbf{v}_p - \mathbf{v}_f) - \frac{\mathrm{d}\mathbf{v}_f}{\mathrm{d}t} + \left(1 - \frac{\rho_f}{\rho_p}\right)\mathbf{g} + \frac{\dot{m}}{m}(\mathbf{v}_f - \mathbf{v}_p). \tag{9}$$

For sufficiently small τ_d (small particles), we can neglect the inertia term on the left hand side of this equation. We also have

$$\frac{\mathrm{d}\mathbf{v}_f}{\mathrm{d}t} = \frac{\partial \mathbf{v}_f}{\partial t} + \mathbf{v}_p \cdot \nabla \mathbf{v}_f.$$

(9) then becomes

$$\mathbf{v}_p = \mathbf{v}_f - \tau_d\left[\frac{\partial \mathbf{v}_f}{\partial t} + \mathbf{v}_p \cdot \nabla \mathbf{v}_f + \left(1 - \frac{\rho_f}{\rho_p}\right)\mathbf{g} + \frac{\dot{m}}{m}(\mathbf{v}_f - \mathbf{v}_p)\right]. \tag{10}$$

Since we know the fluid velocity and acceleration and $\dot{m}$ is given by (7), this is a linear equation for the particle velocity, $\mathbf{v}_p$. This can not only be used to compute the velocity of individual particles for which the inertia can be neglected, but it also makes it possible to write an evolution equation for the particle distribution function.

3.2 Particle Distribution Function

Let $f(\mathbf{r}, m, t)$ be the number density of particles with masses in the range m to $m + \mathrm{d}m$ at $\mathbf{r}$ at time t. Then f satisfies the equation

$$\frac{\partial f}{\partial t} + \nabla \cdot (\mathbf{v}_p f) + \frac{\partial}{\partial m}(\dot{m} f) = s_f, \tag{11}$$

where $\dot{m}$ is given by (7) and s_f accounts for particle splitting or coagulation.

Apart from the source term, (11) is a linear conservation equation for f in which $\dot{m}$ and $\mathbf{v}_p$ are known functions of the fluid state and the particle mass. A conservative numerical scheme can therefore be constructed in precisely the same way as for the continuity equation for the fluid.

Evaporation and condensation lead to a transfer of mass and momentum between the particles and the fluid. The mass source for the fluid is

$$s_\rho = \int_{m_1}^{m_2} m \frac{\partial}{\partial m}(\dot{m} f)\ \mathrm{d}m, \tag{12}$$

where m_1, m_2 are the lower and upper mass limits.

From the equation of motion, (1), the force on the fluid is

$$\mathbf{F} = \int_{m_1}^{m_2} \left[\frac{m}{\tau_d}(\mathbf{v}_p - \mathbf{v}_f) - m\frac{\rho_f}{\rho_p}\mathbf{g} + \dot{m}(\mathbf{v}_p - \mathbf{v}_f) \right] f\ \mathrm{d}m, \tag{13}$$

per unit volume.

The temperature of the particles is determined by equation (6) and is, in general, not equal to that of the fluid. However, if there is no particle splitting or coagulation, then the particle temperature, T_p, only depends upon position, time and particle mass i.e. $T_p = T_p(\mathbf{r}, t, m)$. We therefore also have $e_p = e_p(\mathbf{r}, t, m)$. Conservation of energy and equation (6) give

$$\frac{\partial e_p f}{\partial t} + \nabla \cdot (\mathbf{v}_p e_p f) + \frac{\partial}{\partial m}(\dot{m} e_p f) = f s_e, \tag{14}$$

which is of the same form as (11) and can therefore be dealt with in the same way.

4 Test Calculations

4.1 Individual Particles

Figure 1 shows the total velocity and particles in the Case 1 jet described in the report on the equation of state. Particles were created at regular intervals at four positions: r = 0.1, z = 0.01; r = 0.2, z = 0.01; r = 0.3, z = 0.01; r = 0.4, z = 0.01 with the fluid velocity. The relaxation time was $\tau = 0.002$ and it can be seen that the particles do not follow the streamlines in this case.

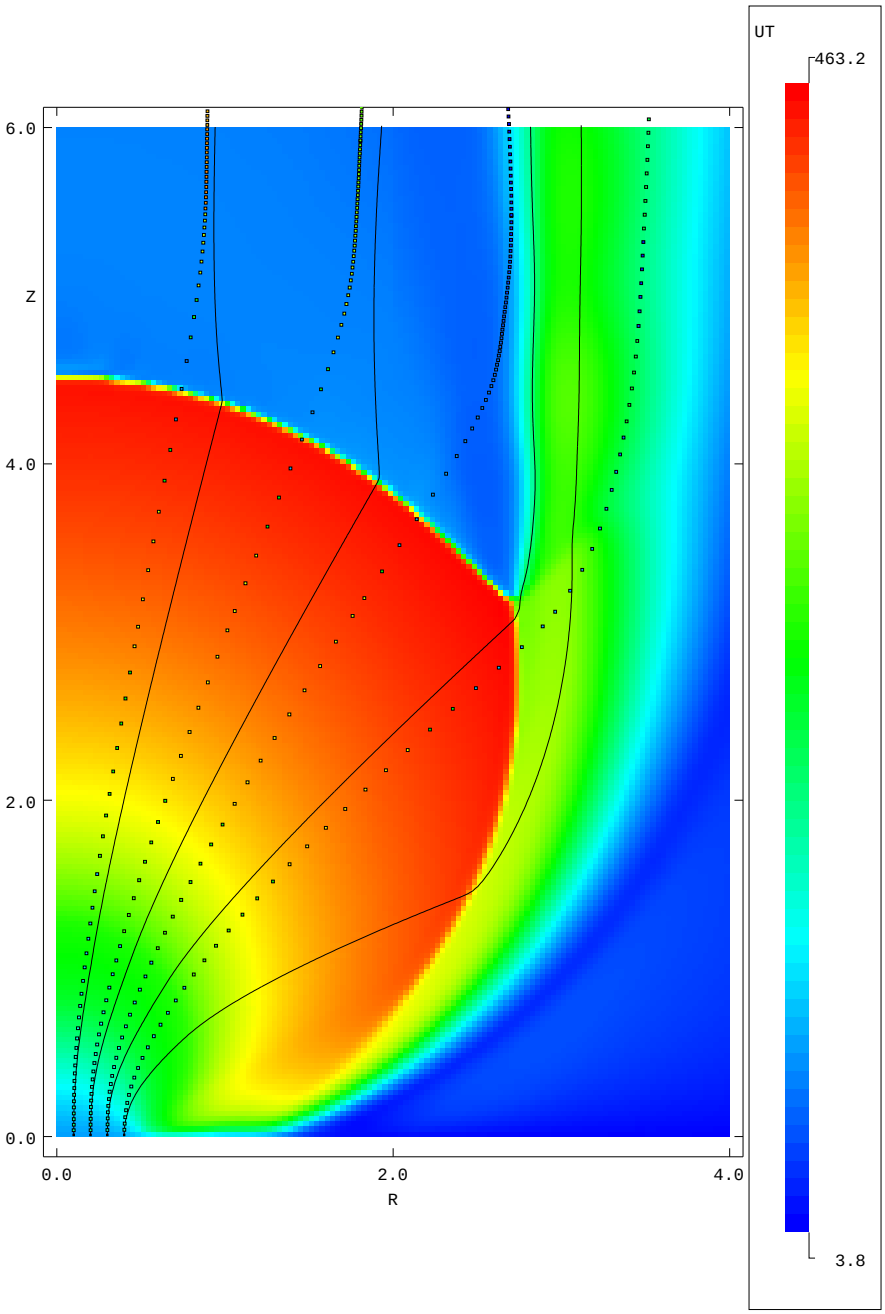


Figure 1: Total velocity, streamlines and particles for the Case 1 jet described in the report on the equation of state.

4.2 Particle Distribution Function

4.2.1 Particle Advection

As a simple test of the particle advection term, we consider a one-dimensional flow with velocity

$$v_f = -\frac{x}{t}. \tag{15}$$

The path of a fluid particle at $x = x_0$ at $t = t_0$ is then

$$x_f = \frac{x_0 t_0}{t}. \tag{16}$$

We have

$$\frac{\partial v_f}{\partial t} + v_p \frac{\partial v_f}{\partial x} = \frac{x}{t^2} - \frac{v_p}{t}$$

so that if we set $\dot{m} = 0$ and neglect gravity, equation (10) gives

$$v_p = -\frac{x/t + \tau_d x/t^2}{1 - \tau_d/t}$$

for the particle velocity.

The particle paths are then given by

$$\frac{\mathrm{d}x_p}{\mathrm{d}t} = v_p = -\frac{x/t + \tau_d x/t^2}{1 - \tau_d/t},$$

which can readily be integrated to give

$$x_p = x_0 \frac{t}{t_0} \left(\frac{t_0 - \tau_d}{t - \tau_d} \right)^2. \tag{17}$$

As expected, (17) reduces to (16) when $\tau_d = 0$ i.e. the particles move with the fluid when the relaxation time is zero.

In this case the evolution equation for the particle distribution function, (11), reduces to

$$\frac{\partial f}{\partial t} + \frac{\partial}{\partial x}(v_p f) = 0,$$

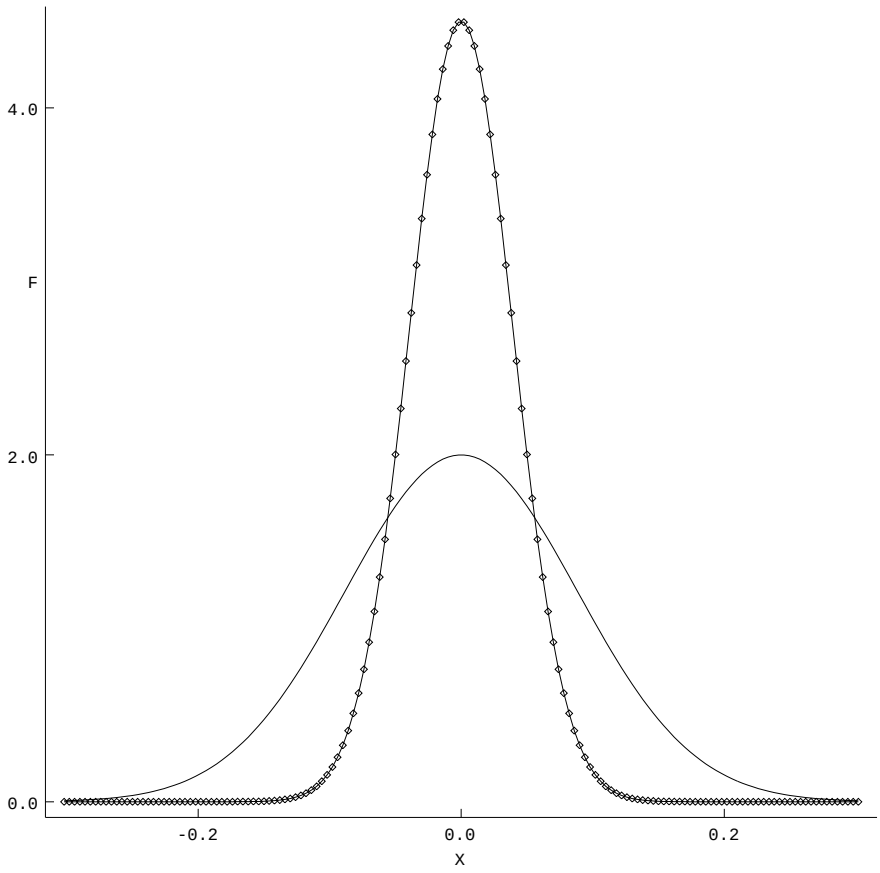


Figure 2: Exact solution at $t = 2$ for $\tau_d = 0.5$ (markers) and $\tau_d = 0$ (line)..

which can be written

$$\frac{\partial f}{\partial t} + v_p \frac{\partial f}{\partial x} = -f \frac{\partial v_p}{\partial x} = f \left(\frac{1/t + \tau_d/t^2}{1 - \tau_d/t} \right).$$

This is a linear hyperbolic equation whose characteristics are the particle paths given by (17). Along these characteristics we have

$$\frac{\mathrm{d}f}{\mathrm{d}t} = f \left(\frac{1/t + \tau_d/t^2}{1 - \tau_d/t} \right).$$

The solution to this is

$$f = f_0(x_0) \left(\frac{t - \tau_d}{t_0 - \tau_d} \right)^2 \frac{t_0}{t}, \tag{18}$$

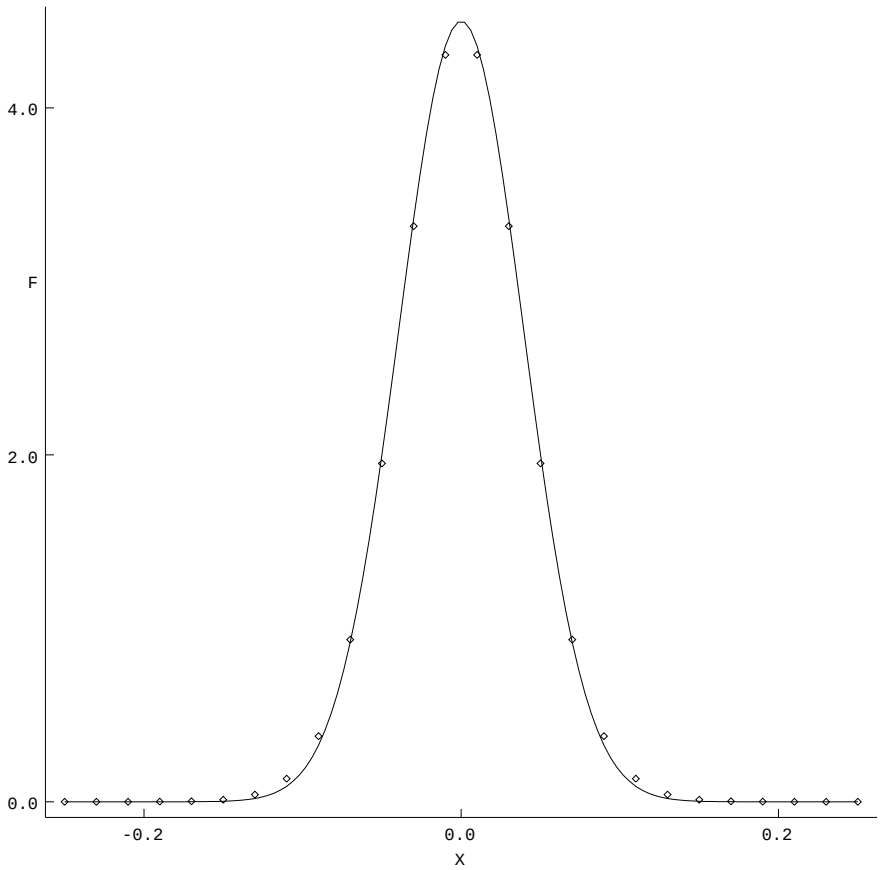


Figure 3: Exact (line) and numerical solution (markers) for a one dimensional test of distribution function advection at $t = 2$ with $\tau_d = 0.5$. The numerical solution had a mesh spacing of 0.02.

where $f_0(x) = f(x, t_0)$ is the initial condition on f and x_0 is the initial position of a particle that is at x at time t. For such a particle, we have from (17)

$$x_0 = x\frac{t_0}{t}\left(\frac{t-\tau_d}{t_0-\tau_d}\right)^2.$$

Substituting this in (18) gives the required solution

$$f = f_0\left[x\frac{t_0}{t}\left(\frac{t-\tau_d}{t_0-\tau_d}\right)^2\right]\left(\frac{t-\tau_d}{t_0-\tau_d}\right)^2\frac{t_0}{t}. \tag{19}$$

For this simple case, the solution at time t can be obtained from that at t_0 by a simple scaling of x and the amplitude. Note that total f is conserved, as it must be. Figure 2

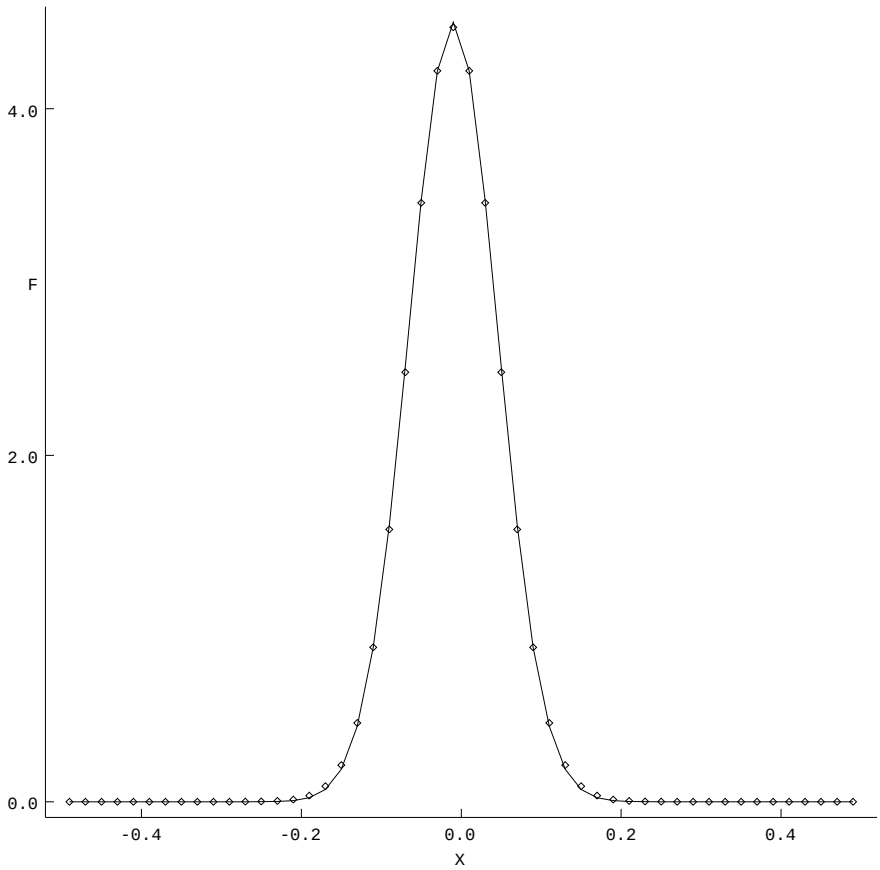


Figure 4: Exact (line) and numerical solution (markers) at $y = 0$ for a two dimensional test of distribution function advection at $t = 2$ with $\tau_d = 0.5$. The numerical solution had a mesh spacing of 0.02.

shows the exact solution at $t = 2$ for an initial Gaussian with a width of 0.25, $t_0 = 1.0$ for $\tau_d = 0$ (particles move with the fluid) and $\tau_d = 0.5$. This shows that the particle velocities are significantly different from the fluid velocity for $\tau_d = 0.5$.

Figure 3 shows a comparison between the numerical and exact solution at $t = 2$ with $\tau_d = 0.5$. It can be seen that the agreement is excellent. Figure 4 shows a comparison between the numerical and exact solution for a two dimensional version of the same case in which the velocities are at 45 degrees to the x-axis. Again, the agreement is excellent.

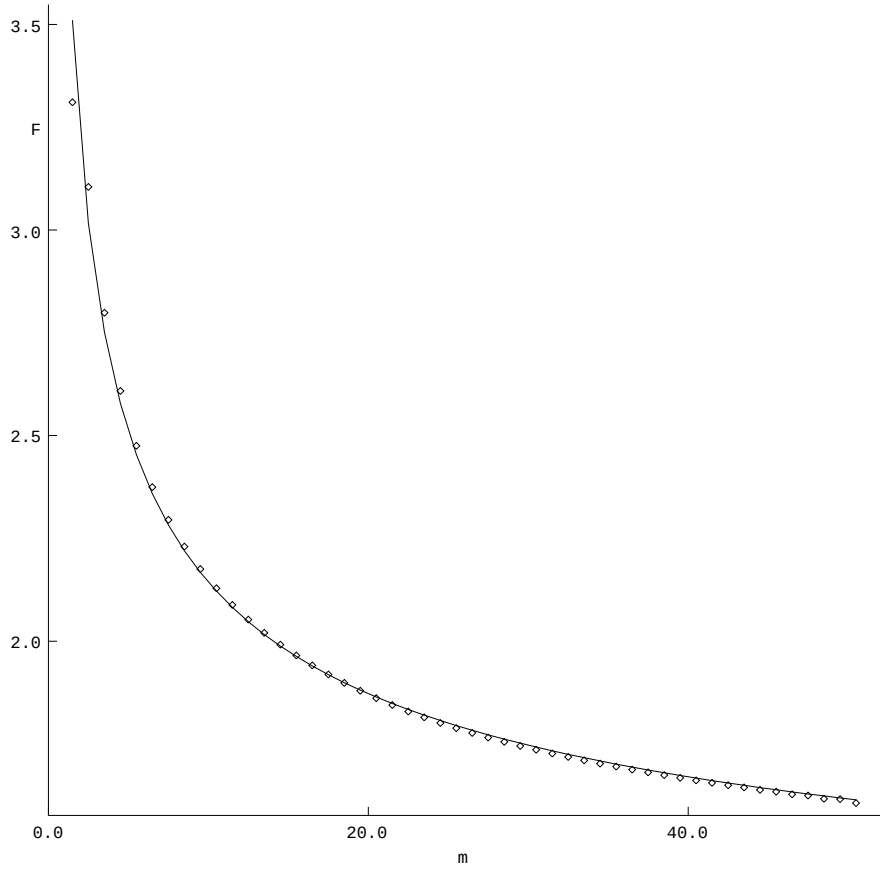


Figure 5: Exact (line) and numerical solution (markers) for a test of distribution function mass evolution at $t = 1$ with $\tau_d = 0.5$. The numerical solution had 50 mass bins.

4.2.2 Particle Mass Change

According to Boyson & Swithenbank, there are cases in which the particle diameter decreases linearly with time i.e.

$$\frac{\mathrm{d}m}{\mathrm{d}t} = -3C_e m^{2/3}, \tag{20}$$

If we neglect advection and set the source terms to zero, then (11) becomes

$$\frac{\partial f}{\partial t} + \frac{\partial}{\partial m}(\dot{m} f) = 0.$$

Using (20), this can be written as

$$\frac{\partial f}{\partial t} + \dot{m}\frac{\partial f}{\partial m} = -f\frac{\partial \dot{m}}{\partial m} = 2C_e m^{-1/3} f. \tag{21}$$

This is a linear hyperbolic equation with characteristics

$$\frac{\mathrm{d}m}{\mathrm{d}t} = \dot{m} = -3C_e m^{2/3}.$$

This is easily integrated and we find that the characteristic through (m, t) intersects the $t = 0$ axis at

$$m_0 = \left(m^{1/3} + C_e t\right)^3. \tag{22}$$

Along the characteristics f satisfies

$$\frac{\mathrm{d}f}{\mathrm{d}t} = 2C_e m^{-1/3} f = \frac{2C_e f}{m_0^{1/3} - C_e t}.$$

This can easily be integrated, and if $f = f_0$ at $t = 0$, we get

$$f(m,t) = f_0(m_0)\left(\frac{m_0}{m}\right)^{2/3} = f_0[(m^{1/3} + C_e t)^3]\left(1 + \frac{C_e t}{m^{1/3}}\right)^2, \tag{23}$$

which is the required solution.

As a test we set $f_0 = 1.0$, $C_e = 1$ and use 50 mass bins in the range $1 \leq m \leq 51$ so that the mass spacing is 1. At the lower and upper mass limits we impose the exact solution.

5 Implementation

The particle model described above was implemented in the AMR code, MG. This section describes the new variables, commands and user functions associated with the model.

5.1 Individual Particles

5.1.1 Variables

The variables for each particle are in a structure, part, which is defined by

```
struct part {
float t, mass, en, xp[MAXD], vp[MAXD], props[MAXPROP];
struct part *nxt, *prev;
struct cell *cptr;
};
```

where

t	time at which particle was last updated
mass	particle mass
en	particle thermal energy
xp[i]	particle coordinate (i = 0 .. nd-1)
vp[i]	particle velocity (i = 0 .. nd-1)
props[i]	other properties (i = 0 .. MAXPROP - 1)

Note that MAXPROP is set in struct.h (MAXPROP = 2) and can be increased if necessary. At present it is assumed that props[0] is the particle density.

Each particle is attached to the cell pointed to by cptr and prev, nxt point to the previous and next particle in the cell (either may be NULL). The start of the cell particle list is pointed to by cptr→pptr.

5.1.2 Commands

Particles are invoked by the command

PARTICLE

and they can be switched off by

NOPARTICLE

The particle command can also be issued after loading an existing model without particles.

5.1.3 User Routines

usrmkpart.c

This creates a particle at xp, assigns it to a cell and returns a pointer to the particle. The

particle position is set, but none of the other properties. The particle is NULL if xp is not on the processor or in the domain.

struct part *usrmkpart(float xp[])

usrprate.c

This should return the time derivatives of the particle variables. It is given

xp[ ]	particle position
vp[ ]	particle velocity
en	particle thermal energy
m	particle mass
pa[ ]	fluid state at xp
props[ ]	particle properties array

and it should return

dvp[ ]	particle acceleration
den	rate of change of particle energy
dm	rate of change of particle mass

void usrprate(float dt, float xp[ ], float vp[ ], float en, float m, float pa[ ], float props[ ], float dvp[ ], float &den, float &dm)

usrpartpop.c

This should be used to update the lagrangian particle population. tcur is the current model time. It is called every time a level is updated.

void usrpartpop(int ilev, float tcur)

usrpartps.c

s returns the value of a particle property in the particle pointed to by pptr. The choice of property is determined by pitem. It returns true if the plot pitem is found, false otherwise.

bool usrpartps(char pitem[ ], struct part *pptr, float &s)

5.1.4 Other Routines

There are a number of utility routines that are not in usr.c.

del_allpart.c

This deletes all particles in the cell pointed to by cptr and sets cptr→pptr=NULL. It is in grid.c.

void del_allpart(struct cell *cptr)

find_fcell.c

Returns a pointer to the finest grid cell that contains the point with coordinates xp. Returns NULL if the cell is not found. ilev is the level on which the cell is found. This can be used to find the cell in which a particle is or should be located.

struct cell *find_fcell(int &ilev, float xp[ ])

rm_part.c

This removes the particle pointed to by pptr from its cell. It does not delete the particle.

void rm_part(struct part *pptr)

If the particle is to be removed entirely, then it is essential to delete the particle by

delete pptr;

5.1.5 Plotting

Particles are plotted by the command

PPART<prop> p

where prop is a particle property. At present the available properties are

mass	particle mass
rho	particle density
en	particle thermal energy
u1	first component of particle velocity
u2	second component of particle velocity
u3	third component of particle velocity

e.g.

plot ppartrho p

plots the particles and colours them according to their density.

If no particle property is found, then the particles are coloured according to their number.

The particles are plotted as coloured markers. The size of these markers can be changed by the command

VRATIO ¡scale¿

where scale is the marker size as a fraction of the size of the plotting domain.

5.2 Distribution Function

5.2.1 Variables

The distribution function and associated variables are stored in the standard qa and pa arrays as described in the user guide as follows:

qa[iqf+i], pa[iqf+i]	f in the ith mass bin (i = 1 .. nbin)
qa[iqfte+i], pa[iqfte+i]	particle thermal energy in the ith mass bin (i = 1 .. nbin)
qa[iqdu0+i], pa[iqdu0+i]	components of $\partial \mathbf{v}_f / \partial t$ (i = 1 .. nd).

The number of mass bins is nbin, the minimum mass is minmass, the maximum mass is maxmass. The mass spacing is

$$\text{dmass} = \frac{\text{maxmass} - \text{minmass}}{\text{nbin}}.$$

5.2.2 Commands

The distribution function is invoked by the command.

DFUN nbin minmass maxmass

The particle internal energy is not calculated by default: it is switched on by the command

DFUNTE

The variables are initialised in usrqis.c in the usual way.

These commands can also be issued after loading an existing model without the distribution function. In that case, usrqis is called for each cell in order to initialise the distribution function variables, while leaving the fluid variables unchanged. This is convenient if one wants to study the behaviour of particles in a model in which the fluid solution has been converged to a steady state.

5.2.3 User Routines

usrtprel.c

This should return the particle relaxation time for a particle with mass pmass. The complete solution is in the pa array.

float usrtprel(float pmass, float pa[])

usrtdmdt.c

This should return the rate of change of particle mass for a particle with mass pmass. The complete solution is in the pa array.

float usrdmdt(float pmass, float pa[])

5.2.4 Plotting

There are two commands for plotting the distribution function: one to plot the space distribution of of a chosen mass bin and one to plot the mass distribution at a particular point in space.

PLOT dfuni p: Plots the space distribution of particles in the ith mass bin. It is entirely analogous to the way in which ordinary scalars are plotted.

DFUNAT x y z: Picks out the point with coordinates x, y, z (it is only necessary to give as many coordinates as there are dimensions). Once this has been set, the following command plots the particle distribution as a function of particle mass at this point.

PLOT pdfun p: Plots the distribution as a function of mass at the point set by the last DFUNAT command. The mass range for plotting can be set by

mr <lower limit> <upper limit>

and the default restored by

nomr

References

[1] Boyson, F., Swithenbank, J., Spray Evaporation in Recirculationg Flow, 17th Symposium (International) on Combustion, Combustion Institute: 443, 1979.

[2] Dodge, L. G., Schwalb, J. A., Fuel Spray Evolution: Comparison of Experiment and CFD Simulation of Nonevaporating Spray, J. Eng. Gas Turbines Power 111, 15, 1989